



INSTITUT POLYTECHNIQUE DE PARIS

Academic Year 2025-2026

First-Year Research Project Report

Numerical Study of the 2D Ising Model

A Markov Chain Monte Carlo Approach

Authors:

Benoît Bala

Mathieu Chazottes

Supervisor:

François Chevoir

(Navier Laboratory)

École nationale des ponts et chaussées

May 18, 2026

Abstract

This report presents a numerical study of the two-dimensional Ising model using Markov Chain Monte Carlo (MCMC) methods. Originally introduced to describe ferromagnetism, the Ising model constitutes a fundamental framework in statistical physics for investigating collective phenomena, critical behavior, and continuous phase transitions. The objective of this work is to reproduce and analyze the critical properties of the model through stochastic simulation techniques, while comparing the efficiency of local and non-local Monte Carlo dynamics.

We first introduce a mathematical formulation of the Ising model and a theoretical framework underlying MCMC methods. We then study the Metropolis algorithm, a standard local-update method, and the Wolff cluster algorithm derived from the link with the Fortuin–Kasteleyn percolation model and the Edwards–Sokal coupling. Particular attention is devoted to the phenomenon of critical slowing down near the critical temperature T_c, where local dynamics become increasingly inefficient.

Using Finite-Size Scaling methods, we estimate several critical exponents from numerical simulations performed on lattices of different sizes. The obtained values are consistent with the exact theoretical values of the two-dimensional Ising model. We also investigate autocorrelation times and dynamic critical behavior in order to compare the performances of the Metropolis and Wolff algorithms. Our results highlight the substantial reduction of critical slowing down achieved by cluster methods, reflected by a significantly smaller dynamic exponent for the Wolff algorithm.

Finally, we briefly discuss computational aspects of the implementation, including GPU parallelization techniques based on checkerboard decomposition for accelerating Metropolis simulations. Overall, this work illustrates how modern numerical methods provide an effective framework for studying critical phenomena in statistical physics.

Keywords: Ising Model, Critical Exponents, Finite-Size Scaling, Markov Chain Monte Carlo, Metropolis Algorithm, Wolff Algorithm, Correlation Time, Dynamic Exponent.

Contents

1 The Ising Model **3**
- 1.1 Mathematical Formulation . . . 3
- 1.2 Phase Transition and Critical Exponents . . . 4

2 Markov Chain Monte Carlo (MCMC) **5**
- 2.1 Theoretical Framework . . . 6
- 2.2 Measuring Physical Observables . . . 7
- 2.3 Metropolis Algorithm . . . 7

3 Cluster Algorithms **9**
- 3.1 Fortuin-Kasteleyn (FK) Percolation . . . 9
- 3.2 Mapping to the Ising Model . . . 10
- 3.3 The Wolff Algorithm . . . 11
- 3.4 Wolff Algorithm Proof of Convergence . . . 13

4 Numerical Results and Comparative Analysis **14**
- 4.1 Finite-Size Scaling (FSS) . . . 14
- 4.2 Correlation Time . . . 17
  - 4.2.1 Estimation of the Correlation Time . . . 17
  - 4.2.2 Dynamic Exponent z . . . 19

5 Conclusion **21**

Appendix A GPU Computing **23**
- A.1 The Checkerboard Metropolis Algorithm . . . 23

Appendix B Note on the Use of AI **25**

Appendix C Complete Source Code **25**

1. The Ising Model

The Ising model, proposed by Wilhelm Lenz in 1920 and analytically solved in one dimension by his student Ernst Ising in 1925, stands as a cornerstone of statistical mechanics. Initially conceived to understand the microscopic origins of ferromagnetism, this model elegantly demonstrates how simple, local interactions can give rise to complex macroscopic phenomena, such as spontaneous magnetization and continuous phase transitions—a feature famously proven for the two-dimensional case by Lars Onsager in 1944 [1].

1.1. Mathematical Formulation

The Ising model describes a system of interacting magnetic dipoles, or "spins", situated on a discrete lattice, where each spin can take the values ± 1 (representing the "up" or "down" states). In our study, we consider the system on a square lattice of size $n \times n$ subject to periodic boundary conditions, effectively forming a discrete torus.

The configuration space, denoted as Λ_n, is the set of all matrices $\mathcal{M}_n(\{-1, 1\})$ indexed over the ring $\mathbb{Z}/n\mathbb{Z}$.

We focus on an elementary formulation of the model characterized by the following properties:

- The external magnetic field is zero.
- Spins interact exclusively with their nearest neighbors.
- The coupling constant (or exchange energy) J is uniform across the lattice, with $J > 0$ corresponding to ferromagnetic interactions.

System energy Let $\sigma \in \Lambda_n$ be a specific spin configuration. The energy of the system is governed by its Hamiltonian $\mathcal{H}$, defined as:

$$\mathcal{H}(\sigma) = -J \sum_{k=1}^{n} \sum_{i=1}^{n} (\sigma_{k,i}\sigma_{k+1,i} + \sigma_{k,i}\sigma_{k,i+1}) = -J \sum_{\langle i,j \rangle} \sigma_i \sigma_j$$

where $\langle i, j \rangle$ denotes the sum over all pairs of nearest-neighbor sites.

Boltzmann-Gibbs Measure At fixed temperature T, the probability distribution $\mathbb{P}_\beta$ (with $\beta = \frac{1}{k_B T}$ [1]) of the system being in a state $\sigma \in \Lambda_n$ at temperature T is given by the Boltzmann weight:

$$\mathbb{P}_\beta(\sigma) = \frac{\exp(-\beta \mathcal{H}(\sigma))}{Z(n, \beta)}$$

Where the normalization constant $Z(n, \beta)$, known as the **partition function**, is the sum over all possible configurations:

$$Z(n, \beta) = \sum_{\sigma \in \Lambda_n} \exp(-\beta \mathcal{H}(\sigma))$$

[1] where k_B is the Boltzmann constant.

1.2. Phase Transition and Critical Exponents

Ferromagnetism is distinguished from paramagnetism by the presence or absence of spontaneous magnetization in a material in the absence of an external magnetic field. The Ising model is a simplified representation of the magnetic behavior of a ferromagnetic material, allowing for the study of the phase transition from a ferromagnetic to a paramagnetic state as a function of temperature T.

Several characteristic quantities of the system are introduced:

- **Mean magnetization** m: For any configuration $\sigma \in \Lambda_n$, it is defined as:

$$m(\sigma) = \frac{1}{n^2} \sum_{i,j=1}^{n} \sigma_{ij}$$

  In the thermodynamic limit ($n \to \infty$), the disordered phase is characterized by a homogeneous distribution of up and down spins, such that m tends toward 0. The system then exhibits paramagnetic behavior. Conversely, in the ordered phase, we observe the emergence of spin clusters with the same orientation. Locally, these clusters generate a non-zero magnetic field.

- **Mean magnetic susceptibility** χ:

$$\chi = \beta n^2 \left(\langle m^2 \rangle - \langle |m| \rangle^2 \right)$$

  χ measures the response of the system's magnetization to a variation in the external magnetic field. The use of the absolute value $\langle |m| \rangle$ is crucial here to handle sign fluctuations in finite-size numerical simulations.

- **Mean specific heat** C_v:

$$C_v = \frac{k_B \beta^2}{n^2} \left(\langle E^2 \rangle - \langle E \rangle^2 \right)$$

  The quantity C_v measures the sensitivity of the system's mean energy to a variation in temperature.

At thermal equilibrium, the system seeks to minimize its free energy $F = E - TS$. Thus, at low temperatures, the system tends toward a minimum energy configuration (an ordered state where all spins are aligned). At high temperatures, the entropic term dominates and the system becomes disordered ($m \to 0$).

Experimentally, one observes the existence of a temperature known as the **Curie temperature** or **critical temperature**, denoted by T_c. Below this temperature, the absolute magnetization $|m|$ of the infinite-volume Ising model is non-zero, whereas above it, it equals 0.

Mathematically, T_c is the unique point of non-analyticity of the free energy $F = -k_B T \ln Z(\beta)$ for the infinite-extension Ising model [2]. Its theoretical expression for a square lattice was

[2] Unlike first-order transitions, the free energy and its first derivative (magnetization) are continuous at T_c. It is the second derivative, the specific heat, that exhibits a singularity.

established in 1941 by physicists Kramers and Wannier [2]:

$$T_c = \frac{2J}{k_B \ln\left(1+\sqrt{2}\right)} \approx 2.269 \frac{J}{k_B}$$

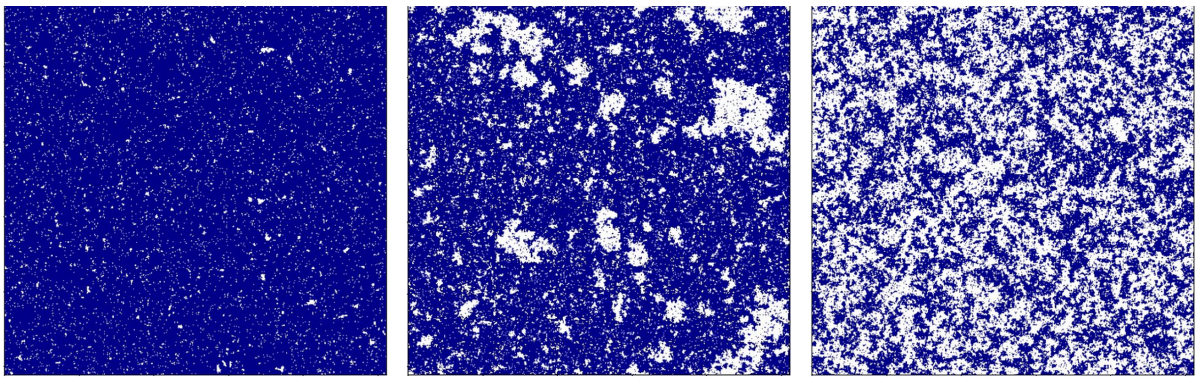


Figure 1: Simulation on a 512×512 system at different temperatures: At $T \approx T_c$, we observe the formation of spin clusters. As T approaches T_c, the size of these clusters increases.[3]

The critical temperature T_c is a singular point near which characteristic quantities follow **scaling laws**:

$$\begin{aligned} \chi &\propto |T - T_c|^{-\gamma} \\ C_v &\propto |T - T_c|^{-\alpha} \\ m &\propto (T_c - T)^{\beta} \quad \text{for } T \to T_c,\ T < T_c \end{aligned}$$

We also introduce the **correlation length** ξ, which characterizes the distance beyond which spins are no longer correlated. It also follows a scaling law: $\xi \propto |T - T_c|^{-\nu}$.

The exponents $\alpha, \beta, \gamma, \nu$ are called **critical exponents**. They are invariant regardless of the lattice geometry or the value of J; this property is known as **universality**. Their theoretical values ($\alpha = 0, \beta = 1/8, \gamma = 7/4, \nu = 1$) stem from Onsager's exact solution (1944) [3].

It turns out that knowing two critical exponents is sufficient to determine the rest through the existence of scaling relations [3]. Their values depend on the model and its dimension. For the Ising Model, it is well-established that in dimensions $d \geq 4$, the system follows **mean-field** behavior (local interactions become negligible compared to the average influence). The 3D Ising model remains an active area of research: there is no exact analytical expression for its free energy or its critical exponents.

2. Markov Chain Monte Carlo (MCMC)

For a lattice of n^2 spins, the configuration space Λ_n contains 2^{n^2} possible states. This exponential growth renders the exact computation of the partition function and macroscopic observables strictly impossible, even for modest system sizes. To overcome this computational wall, we turn

[3] The simulation was performed using the Wolff algorithm (see Section 3.3).

to stochastic sampling techniques. Specifically, we employ Markov Chain Monte Carlo (MCMC) methods to efficiently explore the phase space and generate representative configurations [4].

2.1. Theoretical Framework

Let E be a finite state space and let $(X_k)_{k\in\mathbb{N}}$ be a sequence of random variables taking values in E.

Markov Property The sequence $(X_k)_{k\geq 0}$ is called a **Markov chain** if, for all $k \geq 0$ and for all $e_0, \ldots, e_{k+1} \in E$, the following condition holds:

$$\mathbb{P}(X_{k+1} = e_{k+1} \mid X_k = e_k, \ldots, X_0 = e_0) = \mathbb{P}(X_{k+1} = e_{k+1} \mid X_k = e_k)$$

When the transition probabilities are independent of time k (time-homogeneous chain), we denote by $T(e \to e')$ the transition probability from state e to state e'.

MCMC Methods The goal of a Markov Chain Monte Carlo algorithm for the Ising model is to sample the configuration space Λ_n according to the Boltzmann-Gibbs distribution $\mathbb{P}_\beta$. The principle relies on constructing a Markov chain $(X_k)_{k\geq 0}$ on the state space Λ_n such that its asymptotic distribution converges to the target measure $\mathbb{P}_\beta$:

$$\forall \sigma \in \Lambda_n, \quad \lim_{k\to+\infty} \mathbb{P}(X_k = \sigma) = \mathbb{P}_\beta(\sigma)$$

Definition 1 (Time Homogenous Markov Chain Properties)**.** *Let us now define some properties of time homogenous Markov chains:*

1. ***Stationarity :*** $\mathbb{P}_\beta$ *is a stationary distribution of the Markov chain if it satisfies:*

$$\forall \sigma \in \Lambda_n, \quad \mathbb{P}_\beta(\sigma) = \sum_{\sigma' \in \Lambda_n} \mathbb{P}_\beta(\sigma') T(\sigma' \to \sigma) \tag{1}$$

   This equation is called **global balance**.

2. ***Reversibility (detailed balance) :*** *The Markov chain is reversible for the distribution* $\mathbb{P}_\beta$ *if it satisfies :*

$$\forall \sigma, \sigma' \in \Lambda_n, \quad \mathbb{P}_\beta(\sigma) T(\sigma \to \sigma') = \mathbb{P}_\beta(\sigma') T(\sigma' \to \sigma) \tag{2}$$

   These equations are called **detailed balance**. *If these equations are satisfied, then* $\mathbb{P}_\beta$ *is a stationary distribution.* [4]

3. ***Irreductibility :*** *When any state can be reached from any other state in a finite number of steps with non-zero probability, the Markov chain is irreducible.*

[4] It is often easier to verify the detailed equations for the stationarity.

4. ***Aperiodicity :*** *When the greatest common divisor (GCD) of the lengths of all paths returning to any given state is equal to 1, the Markov chain is aperiodic.*

5. ***Ergodicity :*** *For a finite state space, an irreducible and aperiodic Markov chain is ergodic. In other words, the chain is able to explore the entire state space without getting trapped.*

Theorem 1 (Convergence of MCMC Algorithms)**.** *For a Markov Chain Monte Carlo method to be valid and asymptotically converge to the target distribution $\mathbb{P}_\beta$ from whichever initial distribution, it is sufficient to ensure the chain is ergodic and admits $\mathbb{P}_\beta$ as its unique stationary distribution.*

2.2. Measuring Physical Observables

Regardless of the MCMC method chosen to simulate the Ising model and converge to the target distribution $\mathbb{P}_\beta$, the initial state is selected arbitrarily and is therefore in a non-equilibrium configuration. Thus, it is necessary to run the algorithm long enough to reach thermal equilibrium.

To quantify the execution time of the method, we introduce the unit **MCS** (*Monte Carlo Sweeps*). One MCS corresponds, in the configuration space Λ_n, to attempt flipping n^2 spins—that is trying to sweep the entire grid once [5].

Once the system has reached equilibrium, our goal is to draw independent samples of $\sigma \in \Lambda_n$ according to the measure $\mathbb{P}_\beta$. We then introduce the **autocorrelation time** τ: this is the characteristic time interval required to obtain two statistically independent measurements. It turns out that the behavior of τ strongly depends on the chosen method (see the corresponding section 4.2).

Thus, provided the measurement interval is sufficiently large, the sampled configurations $(\sigma^{(i)})_{i\in\mathbb{N}}$ are (almost-)independent. For any measurable physical quantity Q of the system, according to the ergodic theorem [4], almost surely [6]:

$$\frac{1}{M}\sum_{i=1}^{M} Q(\sigma^{(i)}) \xrightarrow[M\to\infty]{} \langle Q\rangle_{\mathbb{P}_\beta}$$

2.3. Metropolis Algorithm

Introduced in 1953 by Nicholas Metropolis and his colleagues [5] to study interacting particle systems, the Metropolis algorithm is a foundational Markov Chain Monte Carlo method. For the 2D Ising model, it explores the configuration space through sequential, local spin flips. The core of the algorithm relies on a physical acceptance criterion based on the energy change introduced by the proposed flip. If a trial move lowers the system's overall energy, the new configuration is systematically accepted. However, if the flip increases the energy, the move is not strictly rejected; instead, it is accepted with a thermal probability that depends on both the energy penalty and the temperature. This stochastic flexibility allows the system to escape local energy minima,

[5] This process is referred to as **thermalization**: the act of bringing the system to equilibrium with a heat bath at a given temperature.

[6] For the random variable $\sigma \in \Lambda_n$ distributed according to $\mathbb{P}_\beta$, $\langle Q\rangle_{P_\beta} = \mathbb{E}[Q(\sigma)]$. Q is understood as a real-valued function defined on Λ_n.

ensuring that the resulting Markov chain asymptotically samples the correct Boltzmann-Gibbs distribution [6]. The exact process is outlined in the pseudo-code below.

Algorithm 1: Metropolis Algorithm

Input : A configuration $\sigma \in \Lambda_n$, parameters β and J
Output : The updated configuration σ

Select **randomly** a spin s from σ
Let σ' be the configuration obtained from σ by flipping s
if $\mathcal{H}(\sigma') - \mathcal{H}(\sigma) \leq 0$ **then**
    $\sigma \leftarrow \sigma'$ `// Lower energy state: certain acceptance`
else
    Draw a random number r uniformly from $[0, 1]$
    if $r < \exp(-\beta\,(\mathcal{H}(\sigma') - \mathcal{H}(\sigma)))$ **then**
        $\sigma \leftarrow \sigma'$ `// Higher energy state: random acceptance`
return σ

Remark: In practice, we do not recompute $\mathcal{H}(\sigma')$ at each iteration ($\mathcal{O}(n^2)$ cost) but simply calculate the local variation of energy ΔE ($\mathcal{O}(1)$ cost).

Theorem 2. *The Markov chain induced by the Metropolis algorithm is ergodic and admits the Boltzmann-Gibbs measure $\mathbb{P}_\beta$ as its unique stationary distribution.*

Proof. We verify the conditions of Theorem 1.

- **Irreducibility:** From any configuration σ, any other configuration can be reached in a finite number of steps (by flipping spins one by one). Since the probability of accepting each flip is strictly positive, all states in Λ_n communicate. The chain is therefore irreducible.
- **Aperiodicity:** At each step, if the proposed move increases the energy, there is a non-zero probability that it will be rejected. In this case, the system remains in the same state: $T(\sigma \to \sigma) > 0$. The presence of this self-loop guarantees that the chain is aperiodic.
- **Detailed balance:** If σ and σ' differ by more than one spin, the transition probability $T(\sigma \to \sigma')$ is zero by construction, and equation (2) is trivially satisfied ($0 = 0$).

  Let us now consider two states σ and σ' differing by exactly one spin. The transition probability is written as:

  $$T(\sigma \to \sigma') = \frac{1}{n^2} A(\sigma \to \sigma')$$

  where $\frac{1}{n^2}$ corresponds to the probability of choosing the spin in question and $A(\sigma \to \sigma')$ is the acceptance probability.

According to the Metropolis process:

$$A(\sigma \to \sigma') = \begin{cases} \mathrm{e}^{-\beta(\mathcal{H}(\sigma')-\mathcal{H}(\sigma))} & \text{if } \mathcal{H}(\sigma') - \mathcal{H}(\sigma) > 0 \\ 1 & \text{otherwise} \end{cases}$$

We deduce:

$$\frac{T(\sigma \to \sigma')}{T(\sigma' \to \sigma)} = \mathrm{e}^{-\beta(\mathcal{H}(\sigma')-\mathcal{H}(\sigma))} = \frac{\mathbb{P}_\beta(\sigma')}{\mathbb{P}_\beta(\sigma)}$$

□

3. Cluster Algorithms

While the Metropolis algorithm guarantees exact asymptotic sampling, its local update dynamics suffer from **critical slowing down** near the critical temperature T_c. As the correlation length diverges, large macroscopic domains of aligned spins emerge. Local trial flips become highly inefficient at renewing these massive structures, causing the autocorrelation time τ to spike. Overcoming this bottleneck requires shifting to non-local methods, known as **cluster algorithms**, which are designed to flip entire correlated regions in a single step.

To this end, we present an approach based on the exact mapping between the Ising model and percolation theory [7, 8].

3.1. Fortuin-Kasteleyn (FK) Percolation

Bernoulli Percolation Before introducing the FK model, let us consider the classical Bernoulli percolation model with parameter $p \in [0, 1]$. This aims to model the porosity of a material: to this end, consider a grid whose edges represent pipes that are either open or closed with probability p. On a graph (V, E) representing a square lattice of size n, we denote by Ω_n the set of configurations ω. [7]

The probability measure $\mathbb{P}_p$ on Ω_n is:

$$\mathbb{P}_p(\omega) = p^{o(\omega)}(1-p)^{f(\omega)}$$

where $o(\omega)$ is the number of open edges and $f(\omega)$ is the number of closed edges.

FK Percolation The Fortuin-Kasteleyn percolation model [9] (or random-cluster model) weights the connected components (clusters) via a parameter $q > 0$. The probability distribution $\mathbb{P}_{p,q}$ on Ω_n is defined by:

$$\mathbb{P}_{p,q}(\omega) = \frac{p^{o(\omega)}(1-p)^{f(\omega)}q^{k(\omega)}}{Z_{p,q}(\Omega_n)}$$

where $k(\omega)$ is the number of connected components of the graph and:

$$Z_{p,q}(\Omega_n) = \sum_{\omega' \in \Omega_n} p^{o(\omega')}(1-p)^{f(\omega')}q^{k(\omega')}$$

[7] A configuration ω is a function from E to $\{0, 1\}$ (0 = closed, 1 = open). Similarly, one can represent $\sigma \in \Lambda_n$ as a function from V to $\{-1, 1\}$.

This percolation model is connected to the Ising world when $q = 2$, as we shall see. When $q = 1$, we recover Bernoulli percolation.

3.2. Mapping to the Ising Model

The core intuition behind the link between FK representation and the Ising model centers on the concept of information transmission across the lattice.

In the standard Ising model, two neighboring spins may align simply by chance, which does not necessarily imply a transmission of information between them. The FK framework resolves this ambiguity by explicitly modeling the propagation of correlation. It introduces a probabilistic bond between aligned spins, which acts as a mathematical guarantee that their alignment is the direct result of mutual interaction rather than statistical noise.

Consequently, an FK cluster is not merely a geometric grouping of identical spins, but a strictly correlated domain where the state of one spin deterministically dictates the state of the others. The phase transition is thus intuitively reinterpreted as a percolation threshold: the exact point where this network of genuine, underlying correlations spans the entire macroscopic system. *(The rigorous mathematical framework linking these two descriptions, known as the Edwards-Sokal coupling, is detailed in the following section).*

Let C denote the set of pairs $(\sigma, \omega) \in \Lambda_n \times \Omega_n$ compatible through the following coupling:

Let $\sigma \in \Lambda_n$, we define the associated FK percolation model $\omega \in \Omega_n$ as follows: For any edge $e = (i, j) \in E$:

- If $\sigma_i \neq \sigma_j$, then $\omega(e) = 0$.
- If $\sigma_i = \sigma_j$, then $\omega(e) \sim \mathcal{B}(p)$ (Bernoulli distribution).

Let $r(\sigma)$ be the number of edges (x, y) such that $\sigma(x) \neq \sigma(y)$. This process defines a probability distribution $\mathbb{P}_\beta^C$ on C:

$$\begin{aligned}
\mathbb{P}_\beta^C(\sigma, \omega) &= \mathbb{P}_\beta(\sigma) \cdot p^{o(\omega)}(1-p)^{f(\omega)-r(\sigma)} \\
&= \frac{1}{Z(n,\beta)} \exp\left(\beta J \sum_{\langle i,j \rangle} \sigma_i \sigma_j\right) \cdot p^{o(\omega)}(1-p)^{f(\omega)-r(\sigma)} \\
&= \frac{1}{Z(n,\beta)} \exp\left(\beta J \sum_{\langle i,j \rangle} \mathbb{1}_{[\sigma_i=\sigma_j]} - \mathbb{1}_{[\sigma_i \neq \sigma_j]}\right) \cdot p^{o(\omega)}(1-p)^{f(\omega)-r(\sigma)} \\
&= \frac{1}{Z(n,\beta)} \exp\left(\beta J(|A| - 2r(\sigma))\right) \cdot p^{o(\omega)}(1-p)^{f(\omega)-r(\sigma)}
\end{aligned}$$

By choosing $p = 1 - \exp(-2\beta J)$, we obtain after simplification:

$$\mathbb{P}_\beta^C(\sigma, \omega) = \frac{\exp(\beta J|A|)}{Z(n,\beta)} p^{o(\omega)}(1-p)^{f(\omega)}$$

Conversely: Let $\omega \in \Omega_n$. By assigning to each connected component a state ± 1 with probability $1/2$, we define a configuration σ. For $q = 2$, we define a probability distribution $\mathbb{P}^C_{p,q}$ on C:

$$\begin{aligned}\mathbb{P}^C_{p,2}(\omega, \sigma) &= \mathbb{P}_{p,2}(\omega) \cdot \left(\frac{1}{2}\right)^{k(\omega)} \\ &= \frac{p^{o(\omega)}(1-p)^{f(\omega)}}{Z_{p,2}(\Omega_n)}\end{aligned}$$

We deduce that these two distributions $\mathbb{P}^C_\beta$ and $\mathbb{P}^C_{p,2}$ coincide on C under the condition $p = 1 - \mathrm{e}^{-2\beta J}$.

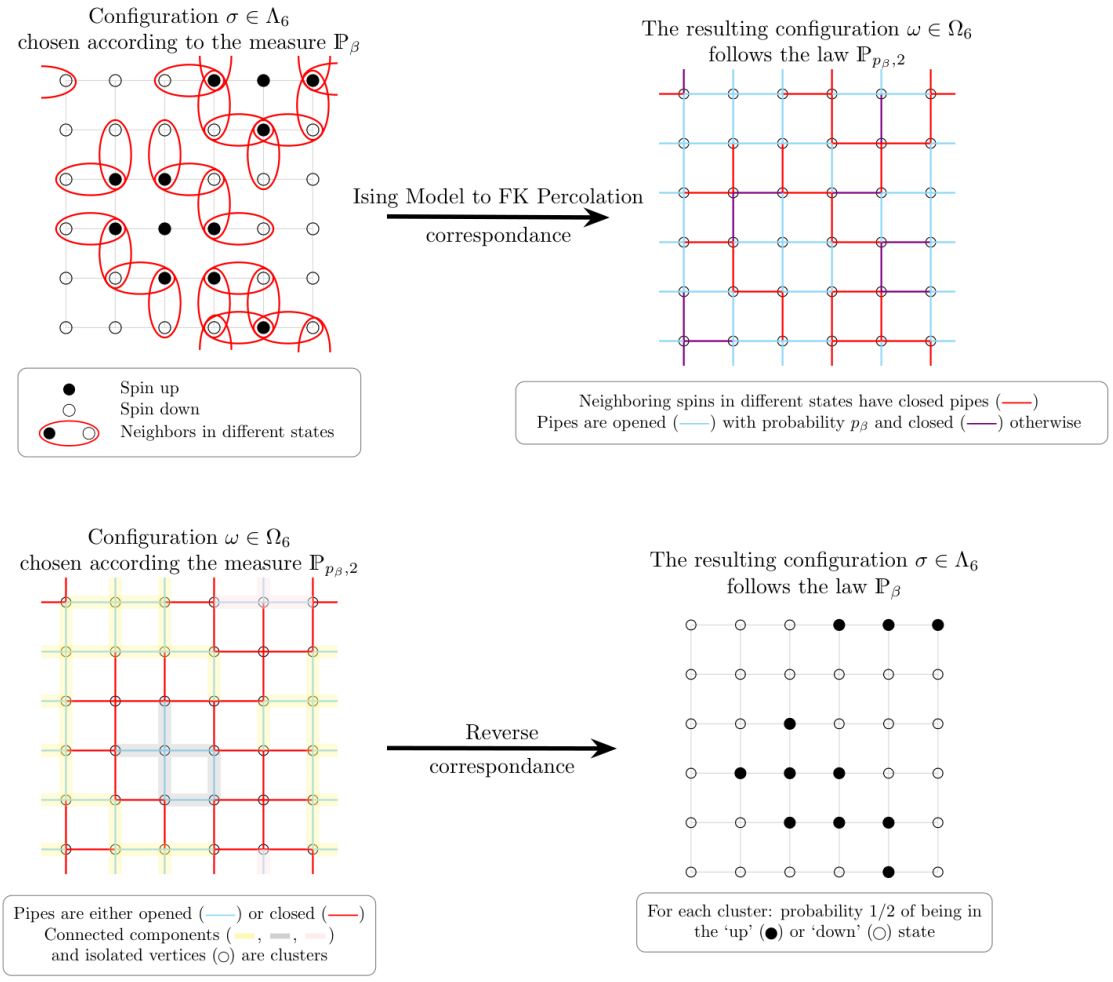


Figure 2: Visualization of Edwards-Sokal coupling.

3.3. The Wolff Algorithm

The Wolff algorithm [6] proposes a cluster-based approach designed to circumvent the problem of **critical slowing down** that affects local methods (such as the Metropolis algorithm, cf. 2.3) near the Curie temperature. The main intuition is as follows: in the vicinity of the phase transition, large domains of strongly correlated spins form. Instead of attempting to flip these spins one by one, the algorithm selects a spin at random and builds around it a connected

component of same-sign spins via a breadth-first traversal. The entire cluster is then flipped simultaneously.

To construct this cluster at inverse temperature β, we define the addition probability of a neighbor to the cluster:

$$\boxed{p_\beta \;=\; 1 - \mathrm{e}^{-2\beta J}}$$

Algorithm 2: Wolff Algorithm

Input : A current configuration $\sigma \in \Lambda_n$, parameters β and J
Output : The updated configuration σ after flipping a cluster

Select a lattice spin s **randomly**
Initialize a queue `to_process` $\leftarrow \{s\}$
Initialize a list `cluster` $\leftarrow \{s\}$
while `to_process` $\neq \emptyset$ **do**
    Pop the spin s' from `to_process`
    foreach neighbor s'' of s' **do**
        `// Check if the neighbor has the same sign AND has not been visited yet`
        if $s = s''$ ***and*** $s'' \notin$ `cluster` **then**
            Draw a random number r uniformly from $[0, 1]$
            if $r < p_\beta$ **then**
                Add s'' to `to_process`
                Add s'' to `cluster`
`// Flip all spins in the cluster`
foreach *site* $s' \in$ `cluster` **do**
    $s' \leftarrow -s'$
return σ

Remarks:

- The cluster construction and flipping rule ensures that the algorithm satisfies detailed balance with respect to the **Boltzmann-Gibbs measure** $\mathbb{P}_\beta$ (cf. 3.4).
- The Wolff method has a natural interpretation within the framework of percolation theory. It amounts to constructing a percolation model associated with the Ising configuration, and then flipping a randomly chosen connected component thereof [8]. It is closely related to other cluster algorithms, such as the Swendsen-Wang algorithm by which it was inspired, and which is a direct translation of the Edwards-Sokal coupling [8].

[8] In practice, however, one obviously does not construct all connected components of the associated percolation model, for efficiency reasons.

3.4. Wolff Algorithm Proof of Convergence

Theorem 3. *The Markov chain induced by the Wolff algorithm is ergodic and admits the Boltzmann-Gibbs measure $\mathbb{P}_\beta$ as its unique stationary distribution.*

Proof. We verify the conditions of Theorem 1.

- **Irreducibility:** Starting from a configuration $\sigma \in \Lambda_n$, for any spin s of σ: one can construct, with a non-zero probability, a Wolff cluster containing only s. Thus, by modifying spins one by one, any configuration σ' in Λ_n can be reached. Consequently, the induced Markov chain is irreducible.

- **Aperiodicity:** With a strictly positive probability, it is possible to return to an initial configuration in 2 steps (by flipping the same cluster of size 1 twice in a row) or in 3 steps (by flipping a spin i, then a neighboring spin j, then the combined cluster $\{i, j\}$). Since $\gcd(2, 3) = 1$, the chain is therefore aperiodic.

- **Detailed balance:**

  Let $T(\sigma \to \sigma')$ be the probability of transitioning from a configuration σ to a configuration σ' via the Wolff process by flipping a specific cluster K.

  To construct this cluster K, the algorithm can choose any of its $|K|$ spins as the initial seed (each having a selection probability of $1/n^2$). Denoting by $L(K)$ the intra-cluster edges and by $L(\sigma, \partial K)$ the edges connecting the spins of the cluster to external spins of the same sign, the transition probability is written as:

  $$T(\sigma \to \sigma') = \frac{|K|}{n^2} p_\beta^{|L(K)|} (1 - p_\beta)^{|L(\sigma, \partial K)|}$$

  For the reverse transition $\sigma' \to \sigma$, the cluster K has been flipped but its size $|K|$ and its internal edges $L(K)$ are unchanged. The algorithm thus has the same probability $|K|/n^2$ of selecting a valid seed. Only the boundary edges of the same sign differ:

  $$T(\sigma' \to \sigma) = \frac{|K|}{n^2} p_\beta^{|L(K)|} (1 - p_\beta)^{|L(\sigma', \partial K)|}$$

  The ratio of the transition probabilities is therefore:

  $$\frac{T(\sigma \to \sigma')}{T(\sigma' \to \sigma)} = (1 - p_\beta)^{|L(\sigma, \partial K)| - |L(\sigma', \partial K)|}$$

  On the other hand, according to the Boltzmann distribution, we have:

  $$\frac{\mathbb{P}_\beta(\sigma')}{\mathbb{P}_\beta(\sigma)} = \exp\left(-\beta(\mathcal{H}(\sigma') - \mathcal{H}(\sigma))\right)$$

  Let us analyze the product $\sigma_i \sigma_j$ for any pair of neighbors in the lattice:

  – If $i, j \in K$: both spins are flipped, so $\sigma'_i \sigma'_j = (-\sigma_i)(-\sigma_j) = \sigma_i \sigma_j$.

– If $i, j \in \Lambda_n \setminus K$: no spin is flipped, so $\sigma'_i\sigma'_j = \sigma_i\sigma_j$.

– If $i \in K$ and $j \notin K$ (on the boundary ∂K): only σ_i is flipped, so $\sigma'_i\sigma'_j = -\sigma_i\sigma_j$.

Thus, the energy difference is carried entirely by the boundary:

$$\frac{\mathbb{P}_\beta(\sigma')}{\mathbb{P}_\beta(\sigma)} = \exp\left(-2\beta J \sum_{\substack{\langle i,j\rangle \\ i\in K, j\notin K}} \sigma_i\sigma_j\right)$$

By identifying the edges of the same sign and of opposite signs on the boundary, this simplifies to:

$$\frac{\mathbb{P}_\beta(\sigma')}{\mathbb{P}_\beta(\sigma)} = \exp\left(-2\beta J(|L(\sigma, \partial K)| - |L(\sigma', \partial K)|)\right)$$

Since the addition probability p_β of the algorithm was specifically chosen such that $1 - p_\beta = \mathrm{e}^{-2\beta J}$, we obtain:

$$\frac{T(\sigma \to \sigma')}{T(\sigma' \to \sigma)} = \frac{\mathbb{P}_\beta(\sigma')}{\mathbb{P}_\beta(\sigma)}$$

The detailed balance condition (2) is therefore well satisfied.

□

4. Numerical Results and Comparative Analysis

4.1. Finite-Size Scaling (FSS)

The primary constraint of a numerical approach for studying the infinite-extension Ising model is the finite size of the simulated systems: there is no divergence of characteristic quantities in the vicinity of the critical temperature, and the system's critical temperature is size-dependent: $T_c(n)$

Thus, one cannot reliably determine the critical exponents in our models by directly measuring the observables in the vicinity of T_c without introducing approximations that are difficult to quantify (uncertainties in the determination of $T_c(n)$, size of the critical region around $T_c(n)$, etc.).

Finite-Size Scaling (FSS) [10, 6] is a method for extracting critical exponents based on the assumption [9] that, for a finite system of size n in the vicinity of $T_c(n)$:

$$\xi \sim n$$

[9] This assumption relies on subtle theoretical arguments originating from renormalization group theory [6].

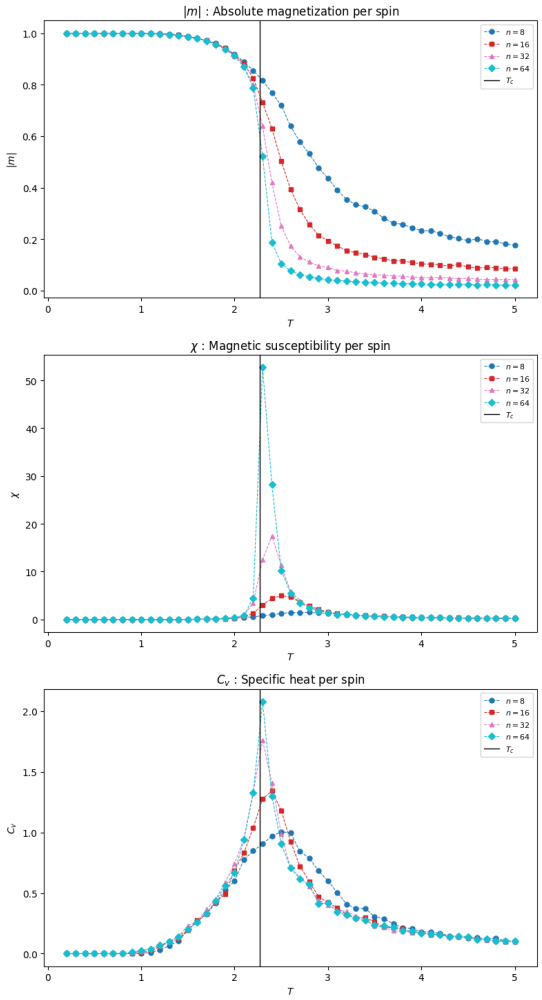


Figure 3: Evolution of Ising model observables (namely absolute magnetization, magnetic susceptibility and specific heat per spin) for different system sizes.

Method:
The curves were obtained by applying the Wolff algorithm to Ising models of sizes $n \in \{8, 16, 32, 64\}$ for $T \in [0.1, 5]$. The measured quantities (or observables) are the energy E and the magnetization m. At a given temperature T, the method consists of :

- **Thermalisation :** 200 MCS [a] to bring the system to equilibrium at temperature T
- **Decorrelation :** To ensure statistically independent measurements, we perform 5 MCS [b] between successive samples.
- **Observables :** After a sufficient number of measurements (here, 1000), we extract, following Sec. 2.2:

  $\langle E \rangle_{\mathbb{P}_\beta}, \langle E^2 \rangle_{\mathbb{P}_\beta}, \langle |m| \rangle_{\mathbb{P}_\beta}, \langle m^2 \rangle_{\mathbb{P}_\beta}$.

Analysis:
We observe that the phase transition manifests itself in the χ and C_v plots through the emergence of a peak in the vicinity of $T_c(n)$. As n increases, the peak becomes sharper and $T_c(n)$ converges toward the theoretical critical temperature T_c. In the $|m|$ plot, the transition is characterized by a drop in $|m|$ beyond $T_c(n)$: the model accurately reproduces the ferromagnetic–paramagnetic transition.

[a] See Section 4.2 for further details
[b] 5 MCS $\gg \tau_{\text{Wolff.}}$, see Section 4.2

For sufficiently large n, $|T_c(n) - T_c| \ll \frac{J}{k_b}$. In practice, this allows us to perform our measurements directly at the theoretical critical temperature T_c. Substituting this equivalence into the scaling laws yields:

$$m \sim n^{-\beta/\nu}$$
$$C_v \sim n^{\alpha/\nu}$$
$$\chi \sim n^{\gamma/\nu}$$

To estimate the critical exponents β and γ [10], it is then sufficient to perform a regression of the logarithm of the chosen observable at the critical temperature as a function of $\ln(n)$.

[10] Estimating ν via FSS requires an estimate of ξ, which is not straightforward and proves computationally expensive. Other methods exist for determining critical exponents, including a direct renormalization group approach [6].

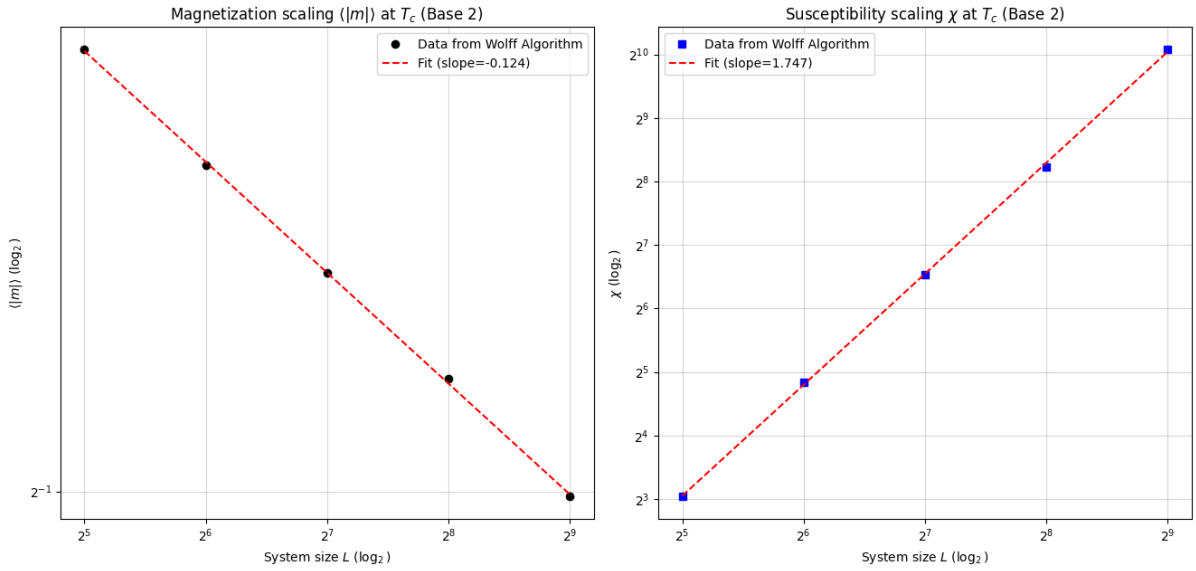


Figure 4: Finite-size scaling (FSS) method applied to the determination of the critical exponents β/ν and γ/ν.

Method:

We performed successive measurements at the critical temperature T_c for systems of sizes $n \in \{32, 64, 128, 256, 512\}$. The measurements were carried out using the same thermalization interval and the same spacing between successive samples, namely 200 MCS and 5 MCS respectively, with the Wolff algorithm.

The experimental critical exponent β (respectively γ) is obtained by computing the slope of the linear regression of $\ln\langle|m|\rangle$ (respectively $\ln\chi$) as a function of $\ln(n)$.

The coefficient α is in fact a special case. Theoretically, $\alpha = 0$, and in the vicinity of T_c, C_v appears to diverge. In reality, the divergence does occur, but not as $\frac{1}{|T-T_c|^\alpha}$; rather, it follows a $-\ln|T - T_c|$ behavior. The corresponding scaling relation is therefore: $C_v \sim C_0 \ln n$ [10].

Uncertainty Estimation :

Since our objective is to estimate the critical exponents, we computed the uncertainties directly on the estimated exponents rather than on the measured observables, in order to avoid error propagation. The uncertainties were evaluated using a *bootstrapping* method [6]:

- Starting from N initial measurements, we construct B new samples of size N by sampling with replacement.
- For each sample, the associated critical exponent is computed using the FSS method.
- The final uncertainty is then defined as the standard deviation of the B computed exponents.

Applying the bootstrapping method ($N = 5000$, $B = 1000$), we obtain (see Figure 4):

$$\frac{\beta}{\nu} = 0.124 \pm 0.001$$
$$\frac{\gamma}{\nu} = 1.75 \pm 0.02$$

Those results are consistent with the theoretical values, namely $\frac{\beta}{\nu} = 0.125$ and $\frac{\gamma}{\nu} = 1.75$. A posteriori, the working assumption underlying the FSS approach is therefore validated. As mentioned in Section 1.2, knowledge of β and γ is sufficient to determine the other critical exponents through scaling relations.

4.2. Correlation Time

Configurations generated sequentially by MCMC simulation algorithms are highly correlated. To accurately evaluate the statistical uncertainties of measured observables (such as the absolute magnetization $|m|$ or energy E) and to compare the efficiency of different algorithms, it is imperative to quantify this temporal correlation. The objective is to estimate the (characteristic) correlation time τ, introduced in 2.2, to determine the thermalization time and the sampling parameters of our algorithms.

A rough initial estimate of the relaxation time towards equilibrium consists of measuring E and $|m|$ at each step of the algorithm and estimating the time required before these quantities fluctuate around a constant value. To avoid being trapped in a local equilibrium, it is useful to repeat this process and take the maximum observed relaxation time as the characteristic value [6].

4.2.1 Estimation of the Correlation Time

For a more rigorous definition to quantify τ, we must examine the **time autocorrelation functions** of the measured quantities. In the following, the absolute magnetization $|m|$ will be used as the reference observable.

Definition 2 (Time Autocorrelation Function)**.** *Let $\mathcal{O}$ be a macroscopic observable measured at each time step t of the simulation, where the standard unit of time is the Monte Carlo Sweep* [11]*. For a time series of N measurements $\{\mathcal{O}_1, \mathcal{O}_2, \ldots, \mathcal{O}_N\}$, the empirical autocovariance $C(t)$ is defined by* [12]*:*

$$C(t) = \frac{1}{N-t} \sum_{i=1}^{N-t} \left(\mathcal{O}_i - \langle \mathcal{O} \rangle\right) \left(\mathcal{O}_{i+t} - \langle \mathcal{O} \rangle\right)$$

where $\langle \mathcal{O} \rangle = \frac{1}{N} \sum_{i=1}^{N} \mathcal{O}_i$ is the sample mean. The normalized autocorrelation function is then given by:

$$\rho(t) = \frac{C(t)}{C(0)}$$

When $\rho(t) \ll 1$, it implies that configurations i and $i + t$ are (virtually) uncorrelated. In practice, the convergence to 0 is fast enough to restrict t to values such that $t \ll N$, making $\rho(t)$ well-defined.

[11] For the Metropolis algorithm, this corresponds to n^2 steps of the algorithm. For the Wolff algorithm, we chose the convention [6] : $t = t_{\text{wolff}} \frac{<L>}{n^2}$ where t_{wolff} is the number of Wolff steps and $< L >$ is the average cluster size.

[12] From a computational perspective, the direct algorithmic calculation of $C(t)$ has a complexity of $\mathcal{O}(N^2)$. To overcome this inefficiency for series containing millions of data points, it is possible to perform the computation using the Fast Fourier Transform (FFT) [6]. This optimization reduces the complexity to $\mathcal{O}(N \log N)$.

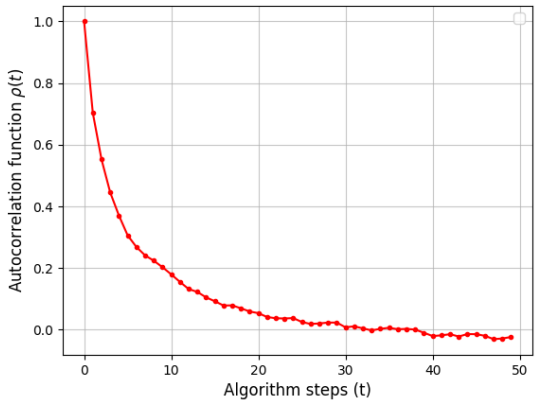


Figure 5: Evolution of the autocorrelation function for $n = 128$ with the Wolff algorithm.

One can observe on Figure 5 that as the iterations progress, we observe a decay of the autocorrelation functions, which tend towards 0.

Assuming the autocorrelation function decays exponentially, the correlation time τ can be defined as the characteristic time of the asymptotic decay of the autocorrelation function at long times [6]:

$$\rho(t) \sim \exp\left(-\frac{t}{\tau}\right) \quad \text{for } t \to \infty$$

This is also called the **exponential correlation time**, denoted by τ_{exp}. It allows us to estimate the number of steps required to reach equilibrium after the system is initialized. In practice, τ_{exp} is estimated by performing a linear regression of $\ln(\rho(t))$ against t, as shown in Figure 6.

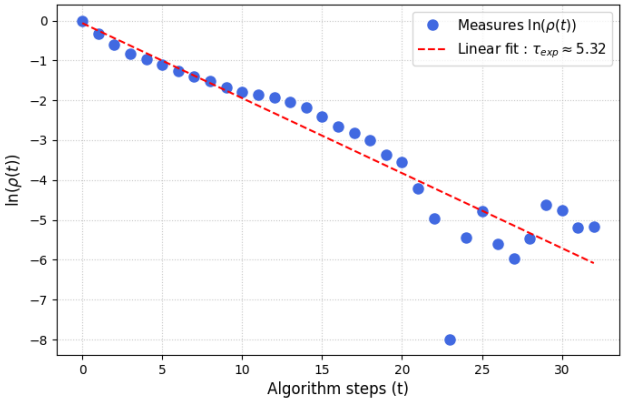


Figure 6: Estimation of τ_{exp} for $n = 128$ with the Wolff algorithm.

It can be observed that beyond a certain time, the regression fit becomes poor. This is due to measurement noise. Therefore, this is a rough estimate of τ_{exp} which depends on the chosen acceptable time interval.

Thus, to obtain uncorrelated samples, the measurement interval must be at least 2τ to ensure $\rho \ll 1$.

The **integrated correlation time** is the quantity that specifies the number of steps required to obtain a new, statistically independent configuration once equilibrium is reached. It is formally defined as:

$$\tau_{\text{int}} = \frac{1}{2} + \sum_{t=1}^{\infty} \rho(t)$$

Intuitively, the formula stems from the numerical approximation of τ, defined as the integral from 0 to ∞ of ρ [6]. Since the true value of $\rho(t)$ vanishes at long times, the empirical estimation of the sum accumulates statistical noise (the variance of the $\rho(t)$ estimator being on the order of $1/N$). Summing to infinity would therefore cause the error on the τ_{int} estimate to diverge. The series must consequently be truncated. The standard method is the use of an "automatic windowing" approach [11]: the sum is stopped at a time $t_{\max}$ such that $t_{\max} \geq c \times \tau_{\text{int}}(t_{\max})$, where the constant c is typically set around 5.

The number of effectively independent measurements generated by the algorithm is thus reduced to $N_{\text{eff}} \approx N/(2\tau_{\text{int}})$.

Although they represent slightly different quantities, in practice, we can consider $\tau_{\text{exp}} \approx \tau_{\text{int}}$ [11]. Thus, to estimate the autocorrelation time τ, we typically use τ_{int}, which can be computed programmatically using the formula above.

4.2.2 Dynamic Exponent z

Empirically, in the vicinity of the critical temperature T_c, we observe a phenomenon known as critical slowing down (CSD) of the numerical methods employed. In fact, τ obeys a scaling law. Near T_c:

$$\tau \propto |T - T_c|^{-z\nu}$$

where z is the dynamic exponent.

As mentioned in 2.2, the correlation time, and consequently z, depend on the MCMC method chosen to simulate the model [13], hence its dynamic nature. Nevertheless, z remains invariant with respect to J and the lattice geometry.

According to the Finite-Size Scaling (FSS) hypothesis for finite-size systems:

$$\tau \sim n^z$$

To estimate this exponent, we successively performed a series of absolute magnetization measurements at the critical temperature T_c, for systems of different sizes n. For each size, the correlation time τ_{int} was estimated.

The uncertainties were obtained using the bootstrapping method introduced in 4.1. However, a modification was made to adapt to the nature of our data: standard bootstrapping assumes independent draws, whereas our time series measurements are inherently highly correlated since they are recorded at each MCS. To avoid underestimating the statistical error, we employed the

[13] Mathematically, τ is actually related to the spectral radius of the stochastic matrix associated with the Markov process of the chosen method.

block bootstrap method [12]. Instead of drawing individual measurements, blocks of l consecutive measurements are randomly sampled. By choosing $l \gg \tau$, the natural temporal correlation within each block is preserved, while the blocks themselves can be considered statistically independent.

When estimating the dynamic exponent for the Metropolis algorithm, the issue is that at $T = T_c$, due to the aforementioned critical slowing down, a massive number of steps is required to obtain independent measurements because the dynamics converge very slowly. To be able to perform simulations within a reasonable time frame, we implemented a variant sometimes called the "Checkerboard Metropolis Algorithm" [13]. This involves cleverly dividing the grid into a checkerboard pattern with two colors, and processing the cells of each color in parallel to accelerate the process (see A.1 for more details).

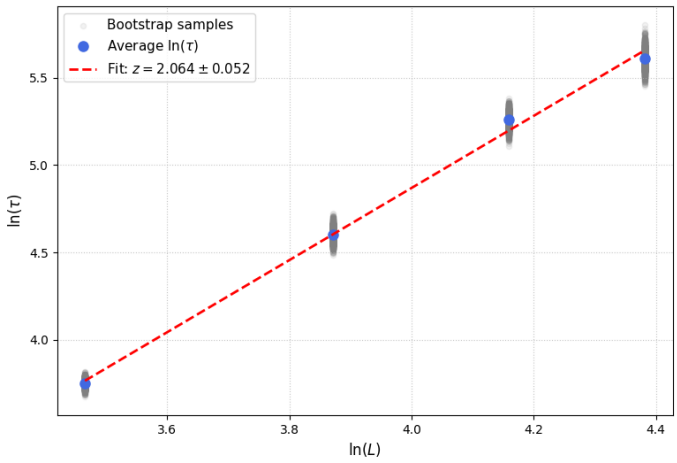


Figure 7: Estimation of the dynamic exponent for the Metropolis algorithm, $n \in \{32, 48, 64, 80\}$.

Using $N = 2 \times 10^6$, $B = 2\,000$, and $l = 40\,000$, we obtained a dynamic exponent of $z = 2.06 \pm 0.05$. This result is consistent with the accepted literature value ($z \approx 2.17$ [6]). The slight underestimation observed can be attributed to the limited range of system sizes (n) that could be explored within a reasonable computation time.

We followed the same procedure for the Wolff algorithm, using $N = 15\,000$, $B = 3\,000$, and $l = 200$.

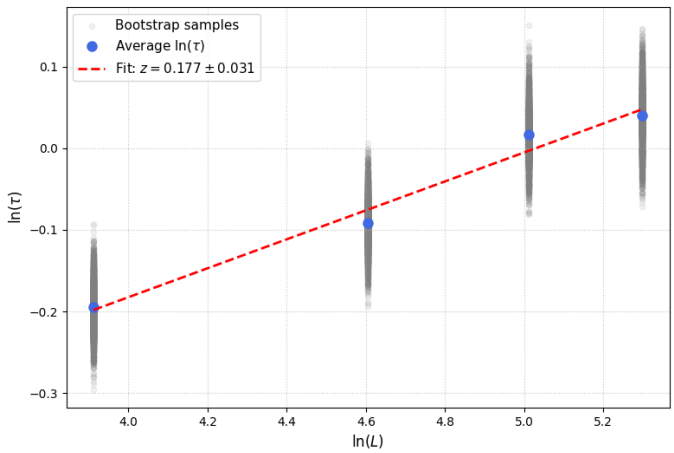


Figure 8: Estimation of the dynamic exponent for the Wolff algorithm, $n \in \{50, 100, 150, 200\}$.

We obtained $z = 0.18 \pm 0.03$, which is quite consistent with the literature (where one generally finds [14] $0.02 \lesssim z \lesssim 0.3$ [14]).

This stark difference between the dynamic exponent of the Metropolis algorithm and that of the Wolff algorithm at the critical temperature perfectly illustrates how the Wolff algorithm circumvents the critical slowing down phenomenon suffered by the Metropolis algorithm. Through its non-local dynamics, it allows for a much more efficient simulation of the Ising model.

5. Conclusion

In this report, we explored the critical phenomena of the two-dimensional Ising model through the lens of Markov Chain Monte Carlo simulations. Our numerical results confirmed the reliability of these stochastic methods in capturing macroscopic thermodynamic behavior—and specifically the continuous ferromagnetic phase transition—arising from simple microscopic interactions. By applying Finite-Size Scaling (FSS) techniques, we estimated the critical exponents β/ν and γ/ν and calculated error bars with bootstrapping technics. The values obtained were consistent with the theoretical values, thereby validating our numerical approach.

A central focus of our study was addressing the computational challenge of critical slowing down near the Curie temperature T_c. While the standard Metropolis algorithm guarantees theoretical convergence to the Boltzmann-Gibbs measure, its local update mechanism renders it highly inefficient in the critical region, where the correlation length diverges. By leveraging the Edwards-Sokal coupling between the Ising model and Fortuin-Kasteleyn percolation , we implemented the Wolff cluster algorithm, which flips entire correlated domains in a single step.

Our autocorrelation analysis quantitatively proved the superiority of this non-local approach: the dynamic critical exponent z was reduced from $z = 2.06 \pm 0.05$ for the Metropolis algorithm to 0.18 ± 0.03 for the Wolff algorithm.

Ultimately, this comparative analysis highlights that modern computational statistical physics relies not only on raw processing power, but fundamentally on the mathematical and physical ingenuity of the sampling algorithms.

[14] The exact estimation of z for the Wolff algorithm is quite subtle and depends heavily on the method used, which explains these variations in results.

References

[1] L. Onsager. Crystal statistics. i. a two-dimensional model with an order-disorder transition. *Physical Review*, 65, 1944.

[2] H. A. Kramers and G. H. Wannier. Statistics of the two-dimensional ferromagnet. part i. *Physical Review*, 60:252–262, 1941.

[3] F. Jedrzejewski. *Modèles aléatoires et physique probabiliste*. Springer, 2009.

[4] M. Lelarge. Chapitre 8 : Chaînes de Markov. Notes de cours, 2009. URL: https://www.di.ens.fr/~lelarge/proba09/ENSmarkov.pdf.

[5] N. Metropolis et al. Equation of state calculations by fast computing machines. *The journal of chemical physics*, 21(6):1087–1092, 1953.

[6] M. E. J. Newman and G. T. Barkema. *Monte Carlo Methods in Statistical Physics*. Oxford University Press, 1999.

[7] W. Werner. *Percolation et modèle d'Ising*. Cours Spécialisés. Société Mathématique de France, 2009.

[8] Y. Velenik. Le modèle d'Ising. Notes de cours, 2009. URL: https://www.unige.ch/~velenik/papers/LN_Ising.pdf.

[9] C.M. Fortuin and P.W. Kasteleyn. On the random-cluster model: I. introduction and relation to other models. *Physica*, 57(4):536–564, 1972.

[10] J. Kotze. Introduction to Monte Carlo methods for an Ising model of a ferromagnet, 2008. arXiv:0803.0217.

[11] A. Sokal. Monte Carlo methods in statistical mechanics: foundations and new algorithms. In *Functional integration: Basics and applications*, pages 131–192. Springer, 1997.

[12] H. R Künsch. The jackknife and the bootstrap for general stationary observations. *The annals of Statistics*, pages 1217–1241, 1989.

[13] K. Yang et al. High performance Monte Carlo simulation of Ising model on TPU clusters. In *Proceedings of the international conference for high performance computing, networking, storage and analysis*, pages 1–15, 2019.

[14] D. Sabogal-Suárez, J.D. Alzate-Cardona, and E. Restrepo-Parra. Static and dynamic critical behavior of thin magnetic Ising films. *Physica A: Statistical Mechanics and its Applications*, 434:60–67, 2015.

A. GPU Computing

To reduce simulation times, we parallelized certain computations on GPU. To this end, we used the Python module Taichi and performed the computations on an AMD Radeon RX 5600 XT graphics card. This allowed us in particular to simulate several thousand Ising grids in parallel to obtain more accurate results, as well as to implement an optimized version of the Metropolis algorithm, presented below.

A.1. The Checkerboard Metropolis Algorithm

To leverage modern parallel computing architectures, such as GPUs, the standard Metropolis algorithm can be adapted using a spatial decomposition technique known as **checkerboard parallelization** [13]. In grid-based systems where interactions are restricted to nearest neighbors, the lattice is partitioned into two independent sub-lattices, analogous to the black and white squares of a checkerboard. Because "black" spins solely interact with "white" neighboring spins, all sites within a single color group are conditionally independent. This allows the algorithm to simultaneously update all "black" spins in one parallel execution pass, followed by a second parallel pass to update all "white" spins. This alternating two-step approach safely preserves the global balance condition of the Markov chain while unlocking important computational speedups.

Theorem 4. *The checkerboard parallelized Metropolis algorithm generates an ergodic Markov chain that converges asymptotically to the Boltzmann-Gibbs measure* $\mathbb{P}_\beta$.

Proof. We verify the conditions of Theorem 1:

Stationarity: We partition our lattice $(\mathbb{Z}/n\mathbb{Z})^2$ into two sub-lattices $\mathcal{W}$ and $\mathcal{B}$, analogous to the white and black squares of a checkerboard. Let $\Lambda_\mathcal{W}$ (resp. $\Lambda_\mathcal{B}$) denote a spin configuration on $\mathcal{W}$ (resp. $\mathcal{B}$).

The nearest neighbors of a site in $\mathcal{W}$ all belong to $\mathcal{B}$, and vice versa. There is no direct interaction between two spins in $\mathcal{W}$, nor between two spins in $\mathcal{B}$.

For any $\sigma_W \in \Lambda_\mathcal{W}$ and $\sigma_B \in \Lambda_\mathcal{B}$, let (σ_W, σ_B) denote the resulting configuration on Λ_n. Then:

$$\mathcal{H}((\sigma_W, \sigma_B)) = -J \sum_{i \in \mathcal{W}} \sigma_{W,i} \left(\sum_{j \in \mathcal{V}(i)} \sigma_{B,j} \right) = -J \sum_{i \in \mathcal{B}} \sigma_{B,i} \left(\sum_{j \in \mathcal{V}(i)} \sigma_{W,j} \right)$$

where $\mathcal{V}(i)$ is the set of nearest neighbors of i. If $i \in \mathcal{W}$, then $\mathcal{V}(i) \subset \mathcal{B}$, and conversely if $i \in \mathcal{B}$.

Let $\sigma_W \in \Lambda_\mathcal{W}$ and $\sigma_B \in \Lambda_\mathcal{B}$. We have:

$$\begin{aligned}
\mathbb{P}_\beta(\sigma_W \mid \sigma_B) &= \frac{\mathbb{P}_\beta(\sigma_W, \sigma_B)}{\sum_{\sigma'_W \in \Lambda_\mathcal{W}} \mathbb{P}_\beta(\sigma'_W, \sigma_B)} \\
&= \frac{\mathrm{e}^{+\beta J \sum_{i \in \mathcal{W}} \sigma_{W,i} \sum_{j \in \mathcal{V}(i)} \sigma_{B,j}}}{\sum_{\sigma'_W \in \Lambda_\mathcal{W}} \mathrm{e}^{+\beta J \sum_{i \in \mathcal{W}} \sigma_{W',i} \sum_{j \in \mathcal{V}(i)} \sigma_{B,j}}} \\
&= \frac{\prod_{i \in \mathcal{W}} \mathrm{e}^{+\beta J \sigma_{W,i} \sum_{j \in \mathcal{V}(i)} \sigma_{B,j}}}{\sum_{\sigma'_W \in \Lambda_\mathcal{W}} \prod_{i \in \mathcal{W}} \mathrm{e}^{+\beta J \sigma_{W',i} \sum_{j \in \mathcal{V}(i)} \sigma_{B,j}}} \\
&= \frac{\prod_{i \in \mathcal{W}} \mathrm{e}^{+\beta J \sigma_{W,i} \sum_{j \in \mathcal{V}(i)} \sigma_{B,j}}}{\prod_{i \in \mathcal{W}} \left(\sum_{\sigma'_{W,i} = \pm 1} \mathrm{e}^{+\beta J \sigma'_{W,i} \sum_{j \in \mathcal{V}(i)} \sigma_{B,j}} \right)} \\
&= \prod_{i \in \mathcal{W}} \frac{\mathrm{e}^{+\beta J \sigma_{W,i} \sum_{j \in \mathcal{V}(i)} \sigma_{B,j}}}{2 \cosh \left(\beta J \sum_{j \in \mathcal{V}(i)} \sigma_{B,j} \right)}
\end{aligned}$$

Thus, we observe that the conditional probability of the event $\bigcap_{i \in \mathcal{W}} [\sigma_i = \sigma_{W,i}]$ given $\bigcap_{j \in \mathcal{B}} [\sigma_j = \sigma_{B,j}]$ is multiplicatively separable in terms of the variables $(\sigma_i)_{i \in \mathcal{W}}$.

Therefore, the random variables $(\sigma_i)_{i \in \mathcal{W}}$ are conditionally independent given the event $\bigcap_{j \in \mathcal{B}} [\sigma_j = \sigma_{B,j}]$. By symmetry, the random variables $(\sigma_i)_{i \in \mathcal{B}}$ are conditionally independent given the event $\bigcap_{i \in \mathcal{W}} [\sigma_i = \sigma_{W,i}]$.

Let T_W (resp. T_B) denote the transition operator of the checkerboard algorithm for all spins in $\mathcal{W}$ (resp. $\mathcal{B}$). For a single algorithm step, we successively apply T_W then T_B.

Let $\sigma = (\sigma_W, \sigma_B)$ and $\sigma' = (\sigma'_W, \sigma'_B) \in \Lambda_n$. According to the checkerboard algorithm process [13]:

$$T(\sigma \to \sigma') = T_W \left((\sigma_W, \sigma_B) \to (\sigma'_W, \sigma_B) \right) \times T_B \left((\sigma'_W, \sigma_B) \to (\sigma'_W, \sigma'_B) \right)$$

By the conditional independence property:

- $T_W \left((\sigma_W, \sigma_B) \to (\sigma'_W, \sigma_B) \right) = \prod_{i \in \mathcal{W}} A(\sigma_{W,i} \to \sigma'_{W,i} \mid \sigma_B)$
- $T_B \left((\sigma'_W, \sigma_B) \to (\sigma'_W, \sigma'_B) \right) = \prod_{j \in \mathcal{B}} A(\sigma_{B,j} \to \sigma'_{B,j} \mid \sigma'_W)$

Therefore, by the stationarity of the local Metropolis algorithm with respect to the measure $\mathbb{P}_\beta$, we deduce:

$$\begin{cases}
\mathbb{P}_\beta(\sigma'_W, \sigma_B) &= \sum_{\sigma_W \in \Lambda_\mathcal{W}} T_W \left((\sigma_W, \sigma_B) \to (\sigma'_W, \sigma_B) \right) \mathbb{P}_\beta(\sigma_W, \sigma_B) \\
\mathbb{P}_\beta(\sigma'_W, \sigma'_B) &= \sum_{\sigma_B \in \Lambda_\mathcal{B}} T_B \left((\sigma'_W, \sigma_B) \to (\sigma'_W, \sigma'_B) \right) \mathbb{P}_\beta(\sigma'_W, \sigma_B)
\end{cases}$$

Combining these equations, we obtain:

$$\mathbb{P}_\beta(\sigma'_W, \sigma'_B) = \sum_{\substack{\sigma_W \in \Lambda_\mathcal{W} \\ \sigma_B \in \Lambda_\mathcal{B}}} T \left((\sigma_W, \sigma_B) \to (\sigma'_W, \sigma'_B) \right) \mathbb{P}_\beta(\sigma_W, \sigma_B)$$

The global balance equation is therefore satisfied.

Irreducibility: There is a strictly positive probability of flipping only a single spin per sub-lattice step. Thus, we can reach any state with a non-zero probability by flipping spins one by one over multiple steps.

Aperiodicity: There is a strictly positive probability of the system remaining in the same state at each step. □

B. Note on the Use of AI

We used LLMs (Gemini, Claude) for the following aspects of our work:

- The aesthetic formatting of the document, and assistance with the creation of TikZ diagrams in LaTeX.
- The English translation of parts of our work initially written in French, as well as the rephrasing of certain paragraphs to improve their clarity.
- The Python code in the Jupyter interface for the implementation of the Ising model, the Metropolis algorithm, the Wolff algorithm, the Bootstrapping method, and FSS was written by us, with AI assistance for debugging coding errors and optimizing certain aspects.
- The implementation of the code to simulate the Ising model on a graphics card was partly generated by AI, in particular to assist with the use of the Taichi module.

C. Complete Source Code

The complete code used for this project is available on a dedicated GitHub repository.

Here is one of the scripts used for estimating the critical exponents with the Wolff algorithm:

```
import numpy as np
import matplotlib.pyplot as plt
from tqdm import tqdm
from collections import deque

## Model implementation

class Modele_Ising:
    def __init__(self, taille, energie_interaction, champ):
        self.N = taille
        self.model = np.random.choice([-1, 1], size=(self.N, self.N))
        self.J = energie_interaction
        self.h = champ
        self.m = np.sum(self.model)
        self.E = self.Energie()

    def Energie(self):
```

```
        s = self.model
        voisins = (
            np.roll(s, 1, axis=0) +
            np.roll(s, -1, axis=0) +
            np.roll(s, 1, axis=1) +
            np.roll(s, -1, axis=1)
        )
        E_interaction = -self.J * np.sum(s * voisins) / 2
        E_champ = -self.h * np.sum(s)
        self.E = E_interaction + E_champ
        return self.E

    def plot(self):
        plt.imshow(self.model, cmap='coolwarm')
        plt.colorbar()
        plt.show()

    def magnetisation(self):
        return self.m / (self.N**2)

    def Metropolis(self, beta):
        i = np.random.randint(0, self.N)
        j = np.random.randint(0, self.N)
        s = self.model[i][j]
        voisins = (
            self.model[(i+1)%self.N, j] +
            self.model[(i-1)%self.N, j] +
            self.model[i, (j+1)%self.N] +
            self.model[i, (j-1)%self.N]
        )
        diffE = 2 * s * (self.J * voisins + self.h)
        if diffE < 0:
            self.m -= 2*self.model[i, j]
            self.E += diffE
            self.model[i, j] *= -1
        else:
            R = np.exp(-beta * diffE)
            p = np.random.uniform(0.0, 1.0)
            if p <= R:
                self.m -= 2*self.model[i, j]
                self.E += diffE
                self.model[i, j] *= -1

## Wolff algorithm

def voisins(i, j, modele):
    return [((i+1)%modele.N, j), ((i-1)%modele.N, j),
            (i, (j+1)%modele.N), (i, (j-1)%modele.N)]
```

```
66
67  def Wolff_algo_etape(modele, beta):
68      # 1.Choose a random seed
69      i_start = np.random.randint(0, modele.N)
70      j_start = np.random.randint(0, modele.N)
71      signe_cible = modele.model[i_start, j_start]
72      proba = 1 - np.exp(-2 * modele.J * beta)
73
74      # 2. Initialize the queue and the cluster
75      a_visiter = deque([(i_start, j_start)])
76      cluster = {(i_start, j_start)}
77
78      while a_visiter:
79          i, j = a_visiter.popleft()
80          for u, v in voisins(i, j, modele):
81              if modele.model[u, v] == signe_cible:
82                  if (u, v) not in cluster:
83                      if np.random.rand() < proba:
84                          a_visiter.append((u, v))
85                          cluster.add((u, v))
86
87      # 3. Flip and update the magnetization
88      taille_cluster = len(cluster)
89      modele.m -= 2 * taille_cluster * signe_cible
90      for (u, v) in cluster:
91          modele.model[u, v] *= -1
92      return taille_cluster
93
94  def thermalisation_W(modele, beta):
95      nb_spin_returned = 0
96      while nb_spin_returned < 200*(modele.N**2):
97          nb_spin_returned += Wolff_algo_etape(modele, beta)
98
99  def mesures_W(modele, beta, pas, nb_mesures):
100     E = np.zeros(nb_mesures)
101     M = np.zeros(nb_mesures)
102     thermalisation_W(modele, beta)
103     for i in range(nb_mesures):
104         nb_spin_returned = 0
105         while nb_spin_returned < pas:
106             nb_spin_returned += Wolff_algo_etape(modele, beta)
107         modele.E = modele.Energie()
108         E[i] = modele.E
109         M[i] = modele.m
110     E_mesure = np.mean(E)
111     M_mesure = np.mean(M/modele.N**2)
112     Msquare  = np.mean((M/modele.N**2)**2)
113     Mabs     = np.mean(np.abs(M)/modele.N**2)
```

```
    Esquare  = np.mean(E**2)
    return E_mesure, M_mesure, Esquare, Msquare, Mabs

def calcul_observables(L, pas, nb_mesures, T_i, T_f, pas_T):
    T = np.arange(T_f, T_i, pas_T)
    resultats = {}
    for taille in L:
        N = taille
        # Call the measurement function over the T range
        E_mes, M_mes, Esquare_mes, Msquare_mes, Mabs_mes = mesures_W_T
           (taille, pas, nb_mesures, T_i, T_f, pas_T)
        beta_arr = 1 / T
        chi = susceptibilite(beta_arr, N, Msquare_mes, Mabs_mes)
        Cv  = capacite_calorifique(beta_arr, N, Esquare_mes, E_mes)
        m   = Mabs_mes
        resultats[taille] = {'T': T, 'm': m, 'chi': chi, 'Cv': Cv}
    return resultats

## FSS and calculation of uncertainties by bootstrapping

Tc_exact = 2 / np.log(1 + np.sqrt(2))

def susceptibilite(beta, N, Msquare, Mabs):
    return (beta * N**2) * (Msquare - Mabs**2)

def capacite_calorifique(beta, N, Esquare, Emean):
    return (beta**2 / N**2) * (Esquare - Emean**2)

def fit_exposant(L_list, observable):
    logL = np.log(L_list)
    logObs = np.log(observable)
    coeffs = np.polyfit(logL, logObs, 1)
    return coeffs[0]

def FFS_incertitudes(L_list, nb_mesures, nb_batchs):
    E_mes = np.zeros((len(L_list), nb_mesures))
    M_mes = np.zeros((len(L_list), nb_mesures))
    for idx_L, taille in tqdm(enumerate(L_list), total=len(L_list)):
        modele = Modele_Ising(taille, energie_interaction=1, champ=0)
        beta = 1 / Tc_exact
        pas = 5*(taille**2)
        thermalisation_W(modele, beta)
        for i in range(nb_mesures):
            nb_spin_returned = 0
            while nb_spin_returned < pas :
                nb_spin_returned += Wolff_algo_etape(modele, beta)
            modele.E = modele.Energie()
            E_mes[idx_L][i] =modele.E
```

```
            M_mes[idx_L][i] =modele.m/(taille**2)

    bootstraping_batchs = np.random.choice(nb_mesures, size=(nb_batchs
        , nb_mesures), replace=True)
    beta_sur_nu = np.zeros(nb_batchs)
    gamma_sur_nu = np.zeros(nb_batchs)
    alpha_sur_nu = np.zeros(nb_batchs)

    for b in range(nb_batchs) :
        chi_tc = np.zeros(len(L_list))
        c_tc   = np.zeros(len(L_list))
        Mabs_tc = np.zeros(len(L_list))
        for idx_L in range (len(L_list)) :
            E_batch = E_mes[idx_L][bootstraping_batchs[b]]
            M_batch = M_mes[idx_L][bootstraping_batchs[b]]
            chi_tc[idx_L] = susceptibilite(beta, L_list[idx_L], np.
                mean(M_batch**2), np.mean(np.abs(M_batch)))
            c_tc[idx_L]   = capacite_calorifique(beta, L_list[idx_L],
                np.mean(E_batch**2), np.mean(E_batch))
            Mabs_tc[idx_L] = np.mean(np.abs(M_batch))
        beta_sur_nu[b] = -fit_exposant(L_list, Mabs_tc)
        gamma_sur_nu[b] = fit_exposant(L_list, chi_tc)
        alpha_sur_nu[b] = fit_exposant(L_list, c_tc)

    sigma_beta = np.std(beta_sur_nu, ddof=1)
    sigma_gamma = np.std(gamma_sur_nu, ddof=1)
    sigma_alpha = np.std(alpha_sur_nu, ddof=1)

    Chi_mes = np.zeros(len(L_list))
    C_mes = np.zeros((len(L_list)))
    Mabs_mes = np.zeros((len(L_list)))
    for idx_L in range (len(L_list)) :
        Chi_mes[idx_L] = susceptibilite(beta, L_list[idx_L], np.mean(
            M_mes[idx_L]**2), np.mean(np.abs(M_mes[idx_L])))
        C_mes[idx_L]   = capacite_calorifique(beta, L_list[idx_L], np.
            mean(E_mes[idx_L]**2), np.mean(E_mes[idx_L]))
        Mabs_mes[idx_L] = np.mean(np.abs(M_mes[idx_L]))

    beta_sur_nu_mes = -fit_exposant(L_list, Mabs_mes)
    gamma_sur_nu_mes = fit_exposant(L_list, Chi_mes)
    alpha_sur_nu_mes = fit_exposant(L_list, C_mes)

    print("beta/nu␣=", beta_sur_nu_mes, "+/-", sigma_beta)
    print("gamma/nu␣=", gamma_sur_nu_mes, "+/-", sigma_gamma)
    print("alpha/nu␣=", alpha_sur_nu_mes, "+/-", sigma_alpha)

    return Chi_mes, C_mes, Mabs_mes, beta_sur_nu_mes, gamma_sur_nu_mes
        , alpha_sur_nu_mes, sigma_beta, sigma_gamma, sigma_alpha
```